



Available at
www.ElsevierComputerScience.com
POWERED BY SCIENCE @ DIRECT®
Journal of Algorithms 50 (2004) 23–48

**Journal of
Algorithms**

www.elsevier.com/locate/jalgor

An $\frac{8}{13}$ -approximation algorithm for the asymmetric maximum TSP[☆]

Markus Bläser

Institut für Theoretische Informatik, Universität zu Lübeck, Wallstr. 40, 23560 Lübeck, Germany

Received 3 May 2002

Abstract

We present a polynomial time approximation algorithm for the asymmetric maximum traveling salesperson problem that achieves performance ratio $\frac{8}{13}(1 - 1/n)$. The running time of our algorithm is $O(n^3)$.

© 2003 Elsevier Inc. All rights reserved.

1. Introduction

The traveling salesperson problem (TSP) is a well-known and fundamental problem in computer science with a lot of practical applications. Since it is NP-hard and also hard to approximate (at least most of its variants), much effort has been spent on designing good approximation algorithms for this problem.

Many of the approximation algorithms and heuristics are concerned with the problem of finding a shortest (i.e., minimum weight) Hamiltonian tour. But here we consider approximation algorithms that try to find a longest (that is, maximum weight) Hamiltonian tour. Moreover, we focus on the case where the weight function is asymmetric, i.e., the weight of the edge from u to v may differ from the weight of the edge from v to u . The asymmetric problem is in general harder than the symmetric version (which is of course a special case of the asymmetric problem). For instance, in the case of finding a minimum weight tour under the triangle inequality, the best approximation algorithm

[☆] A shortened version appeared in the Proc. of the 13th Ann. ACM–SIAM Symp. on Discrete Algorithms (SODA), SIAM, 2002, pp. 64–73.

E-mail address: blaeser@tcs.uni-luebeck.de.

currently known for the symmetric problem achieves an approximation factor of $\frac{3}{2}$ [4], while in the asymmetric case, we only have a factor of $\log n$ [6] (see, however, [3]).

Formally, we are given a complete directed graph G without loops and a weight function w assigning each edge of G a nonnegative weight. Except that each weight is nonnegative, there are no further requirements for w , in particular, w neither needs to be symmetric nor to satisfy the triangle inequality. We call the maximization problem of finding a Hamiltonian tour of maximum weight the asymmetric maximum TSP. This problem is of importance, since a good approximation algorithm for the asymmetric maximum TSP yields a good approximation algorithm for the shortest common superstring problem, which has received quite some attention in the past decade, see, e.g., [2,14–16] and the references therein. The shortest common superstring problem arises in DNA sequence assembly and data compression. The asymmetric maximum TSP is APX-hard. This follows from the hardness of TSP with weights one and two [12] by replacing weights two with weights zero. Thus, there is no PTAS (polynomial time approximation scheme, see, e.g., [1] for a definition) for the asymmetric maximum TSP unless $P = NP$.

1.1. Previous results

The first approximation algorithm for the asymmetric maximum TSP with guaranteed approximation performance is due to Fisher, Nemhauser, and Wolsey [5]. They achieve an approximation factor of $\frac{1}{2}$. They start with computing a maximum weight cycle cover, which can be done in polynomial time by reduction to maximum weight bipartite matching. A cycle cover of a (directed) graph is a set of (directed) cycles such that each node is part of exactly one cycle. Since the problem of finding a maximum weight directed cycle cover is a relaxation of the asymmetric maximum TSP, the weight of an optimum cycle cover is an upper bound for the weight of a maximum weight Hamiltonian tour. In the asymmetric case, each cycle has at least two edges. Thus, after discarding the lightest edge in each cycle, we are left with a node disjoint collection of paths. This collection has at least half the weight of the cycle cover. Since the edge weights are nonnegative, we can patch the paths arbitrarily together to obtain a Hamiltonian tour with at least half the weight of an optimal tour. This result was improved by Kosaraju, Park, and Stein [9] who design a polynomial time approximation algorithm with performance ratio $\frac{38}{63} \approx 0.603$.

In the symmetric case, one can compute a maximum weight cycle cover in polynomial time such that each cycle of the cover has at least three edges. Discarding the lightest edge of each cycle yields a factor $\frac{2}{3}$ -approximation algorithm for the symmetric maximum TSP [5]. The best (deterministic) approximation algorithm currently known in the symmetric case is due to Serdyukov [13] who presented a factor $\frac{3}{4}$ -approximation algorithm. Hassin and Rubinstein [7] designed a randomized algorithm with expected performance ratio $\frac{25}{33} - \epsilon$ for any $\epsilon > 0$. For the case where the weight function also obeys the triangle inequality, Kostochka and Serdyukov [10] even obtained a factor of $\frac{5}{6}$. Quite recently, Hassin and Rubinstein [8] gave a randomized polynomial time approximation algorithm for this problem achieving an approximation factor of $\frac{7}{8}$.

1.2. Our contribution

Our first main result is a polynomial time approximation algorithm for the asymmetric maximum TSP with performance ratio $\frac{8}{13}(1 - 1/n) \approx 0.615$. The running time of our algorithm is $O(n^3)$.

Our second main contribution is the first rigorous proof of the so-called path coloring lemma (Lemma 3), a useful subroutine for computing maximum weight Hamiltonian tours. This lemma was formulated by Kosaraju, Park, and Stein [9], but they never gave a proof of this lemma. Our proof is algorithmic, a coloring as described in the lemma can be found in cubic time. We also present some useful variants of the path coloring lemma (Lemmas 2 and 7).

At the time of discovery, our algorithm achieved the best approximation performance. Meanwhile, Lewenstein and Sviridenko [11] obtained a factor $\frac{5}{8}$ -approximation algorithm for the asymmetric maximum TSP. Our algorithm still remains interesting, because their algorithm bases on an LP formulation, while our algorithm is purely combinatorial. Moreover, our running time of $O(n^3)$ is favorable, since beside solving an LP, Lewenstein and Sviridenko's algorithm requires up to n^2 applications of the path coloring lemma (Lemma 3), each (currently) taking time $O(n^3)$.

A related problem is the computation of maximum weight directed k -cycle covers. A directed k -cycle cover is a directed cycle cover where each cycle has at least k edges. As already mentioned, the problem of finding a maximum weight 2-cycle cover is in P. For $k \geq 3$, the problem of finding a maximum weight k -cycle cover is APX-hard, even if we allow only weights one and two (Bläser and Manthey, in preparation). Our algorithm can be utilized to approximate maximum weight k -cycle covers, just by viewing our Hamiltonian tour as a special case of a k -cycle cover. Up to lower order terms, we obtain approximation factors $\frac{9}{16}, \frac{25}{43}, \frac{25}{43}, \frac{49}{83}, \frac{49}{83}, \dots$ for $k = 3, 4, 5, 6, 7, \dots$. As k grows, the sequence of factors goes to $\frac{8}{13}$. Since for $k > n/2$ the maximum weight directed k -cycle cover problem is exactly the maximum asymmetric TSP, we then achieve approximation ratio $\frac{8}{13}$.

2. The algorithms

In this section, we present four algorithms for the asymmetric maximum TSP. The heaviest of the four tours produced by these algorithms will have at least a fraction of $\frac{8}{13}$ of the weight of an optimum tour. We assume that the input graph G always has an even number of nodes. Otherwise, we add one dummy node and assign each edge entering or leaving this node weight zero. Any tour of weight W in the old graph gives a tour of weight $(1 - 1/n) \cdot W$ in the modified one. Thus, we still achieve an approximation ratio of $\frac{8}{13}(1 - 1/n)$.

We first settle some notations and conventions: for a set of nodes V , let $K(V)$ denote the edge set $(V \times V) \setminus \{(v, v) \mid v \in V\}$. Throughout the present section, we are considering graphs $G = (V, K(V), w)$ where $w : K(V) \rightarrow \mathbb{Q}_{\geq 0}$ is a weight function that attains only nonnegative values. Basically, G is completely determined by the weight function w . Our aim is to find a Hamiltonian tour of maximum weight.

For a subset A of the set of edges of G , let $w(A)$ denote the weight of A , that is, the sum of the weights taken over all edges in A .

In all of the algorithms exhibited in this section, we compute a cycle cover C on the given graph G . In the following, such a cycle cover C is given by the set of edges participating in it. To avoid lengthy repetitions, we fix some names for the remainder of this section: $C_1, \dots, C_\ell$ will denote the cycles of the computed cycle cover C . Let $I_2 \subseteq \{1, \dots, \ell\}$ be the set of all indices i such that C_i is a 2-cycle, that is, a cycle consisting of two edges (and, of course, two nodes). In the same way, let I_3 be the set of all indices that correspond to 3-cycles. Finally, let $I_{\geq 4} := \{1, \dots, \ell\} \setminus (I_2 \cup I_3)$. Set $W = w(C)$.

For $i \in I_2$, choose b_i and c_i from $[0, 1]$ such that $b_i \cdot W$ is the weight of the heavier edge in C_i and $c_i \cdot W$ the weight of the lighter one. Here and in the following, we break ties arbitrarily. The two nodes participating in C_i are denoted by s_i and t_i . We assume that the heavier edge is (s_i, t_i) . Let $b = \sum_{i \in I_2} b_i$ and $c = \sum_{i \in I_2} c_i$. Sometimes we also consider a collection of 2-cycles T (not necessarily a cover). In this case, we use the same notations.

For $j \in I_3$, let $x_j \cdot W$, $y_j \cdot W$, and $z_j \cdot W$ denote the weight of the heaviest edge, the edge in the middle, and the lightest edge, respectively. Define x , y , and z in the same manner as b and c .

Finally, for every $k \in I_{\geq 4}$, let $p_k \in [0, 1]$ such that $p_k \cdot W$ is the weight of the lightest edge in C_k and let $p = \sum_{k \in I_{\geq 4}} p_k$.

An edge e is called *incident* with a node v , if either $e = (v, u)$ or $e = (u, v)$ for some node u . A node v *belongs* to a path or a cycle if there is an edge incident with v in the path or cycle.

2.1. Algorithm 1

Algorithm 1 is a simple $\frac{1}{2}$ -approximation algorithm. Note that in the last step, we cannot loose any weight, because all edge weights are nonnegative. The worst case for Algorithm 1 is $b = c = \frac{1}{2}$. However, if C contains cycles with three or more edges that have a significant portion of the total weight of C or b is much larger than c , then Algorithm 1 obtains a better approximation. More precisely, the weight $w(P)$ contained in P is $(1 - c - z - p) \cdot W$. Plugging in $p \leq \frac{1}{4}(1 - b - c - x - y - z)$, we obtain the following bound on the approximation performance of Algorithm 1.

Theorem 1. *Algorithm 1 returns a $(\frac{3}{4} + \frac{1}{4}b - \frac{3}{4}c + \frac{1}{4}x + \frac{1}{4}y - \frac{3}{4}z)$ -approximation to a Hamiltonian tour of maximum weight in G .*

Input: a graph $G = (V, K(V), w)$.

Output: a Hamiltonian tour on G .

- (1) Compute a maximum weight cycle cover C of G .
- (2) Discard the lightest edge of each cycle in C to obtain a collection of node disjoint paths P .
- (3) Construct a tour by patching the paths in P arbitrarily together.

Algorithm 1.

2.2. Algorithm 2

The main idea of Algorithm 2 is to pay special attention to the 2-cycles of the cover, as already done by Kosaraju, Park, and Stein [9]. We present a refined approach which is superior to the one by Kosaraju, Park, and Stein when b is larger than c .

A *matching* of a directed weighted graph $(V, K(V), w)$ is a set of edges $M \subseteq K(V)$ such that for all pairs of edges $(u, v), (x, y) \in M$, the nodes u, v, x, y are pairwise distinct. M is called *perfect*, if for each node $v \in V$, there is an edge $e \in M$ incident with v . A matching M is a *matching of maximum weight*, if $w(M) \geq w(M')$ for all other matchings M' . Note that for our purposes, we can always assume that a matching of maximum weight is perfect, since we assumed that the number of nodes of G is even and w attains only nonnegative values. We can compute a matching of maximum weight in a directed weighted graph in time $O(n^3)$ by reduction to the undirected case: we replace the two edges between each pair of nodes by an undirected one with weight equal to the maximum of the weight of the two replaced edges, then compute a matching of maximum weight on that graph, and then translate everything back into the directed world by replacing each undirected edge by the directed one for which the maximum was attained (breaking ties arbitrarily).

If $e = (u, v)$ and $f = (x, y)$ are two edges in a directed graph, e and f are called *compatible with each other*, if $v = x$ and $u \neq y$ or $y = u$ and $v \neq x$. In other words, we may patch e and f together and obtain a path with two edges.

The idea of Algorithm 2 is to combine a cycle cover and a matching on G . We will use two different weight functions in Algorithm 2: the original function w and a modified

Input: a graph $G = (V, K(V), w)$.

Output: a Hamiltonian tour on G .

- (1) Compute a maximum w -weight cycle cover C of G .
- (2) Define a new weight function w' on $K(V)$ as follows. For all $i \in I_2$, set $w'((s_i, t_i)) = w'((t_i, s_i)) = 2(b_i - c_i) \cdot W$. For all $i, j \in I_2, i \neq j$, set $w'((t_i, s_j)) = w((t_i, s_j)) + (b_i - c_i) \cdot W + (b_j - c_j) \cdot W$. For all $i \in I_2$, set $w'((u, s_i)) = w((u, s_i)) + (b_i - c_i) \cdot W$ and $w'((t_i, v)) = w((t_i, v)) + (b_i - c_i) \cdot W$, if $u \notin \{t_k \mid k \in I_2\}$ and $v \notin \{s_k \mid k \in I_2\}$. For all other edges e , set $w'(e) = w(e)$.
- (3) Compute a perfect matching M of maximum w' -weight on $(V, K(V), w')$.
- (4) Discard the lightest edge of each cycle in C that has at least three edges. This yields a collection of node disjoint paths P .
- (5) Let T be the collection of all 2-cycles from C that do not share an edge with M . (In other words, M did not pick one of the two edges of those 2-cycles.) Let $\hat{M}$ be the set of all edges from M that do not share an edge with a 2-cycle from C .
- (6) Let H be the multigraph obtained from $(V, T \cup P \cup \hat{M})$ by contracting the cycles in T . Color the multigraph H according to Lemma 2 and transfer the coloring back to $(V, P \cup \hat{M})$.
- (7) For each 2-cycle not in T , add the heavier edge to both color classes. For each 2-cycle C_i from T , let e_i and f_i be the two edges from $\hat{M}$ incident with s_i and t_i . If both e_i and f_i have the same color, then add the compatible edge of C_i to both color classes. Otherwise, add the edge of C_i compatible to e_i to the color class of e_i and the edge compatible to f_i to the color class of f_i .
- (8) Patch together the paths of the color class with larger w -weight to obtain the Hamiltonian tour.

Algorithm 2.

one w' . If it is not clear from the context to which one we refer, we will speak of the w -weight and w' -weight of a Hamiltonian tour, cycle cover, etc., to clarify the case. In a nutshell, Algorithm 2 works as follows: it also starts with computing a cycle cover C of maximum w -weight. Then we define a new weight function w' as follows: for each 2-cycle C_i in C , we change the weight of both edges to $2(b_i - c_i) \cdot W$ and add an amount of $(b_i - c_i) \cdot W$ to the weight of each edge compatible with the heavier edge (s_i, t_i) of C_i . These correction terms are needed to cope with unbalanced 2-cycles. Next we compute a maximum w' -weight matching M on G . Since a Hamiltonian tour can run through only one edge of each 2-cycle, it can be shown that M has w' -weight at least $\frac{1}{2}(L + (2b - 3c) \cdot W)$, where L is the w -weight of an optimal Hamiltonian tour. For each cycle in C with at least three edges, we discard the lightest edge to obtain a collection P of node disjoint paths. We then use Lemma 2 to color the edges from M and P with two colors such that the edges of each color class form a collection of node disjoint paths in such a way that we can add one (not necessarily the same) edge from each 2-cycle to both color classes and still have two classes of node disjoint paths. Patching the paths of the class with larger w -weight together gives the Hamiltonian tour of Algorithm 2.

The reason for the modification of the weights is to take advantage of very unbalanced 2-cycles, since in Algorithm 3, we modify the given cover C in such a way that it (hopefully) contains very unbalanced 2-cycles and then run Algorithm 2 on the modified cover. The weights are modified in such a way that we can switch back to the old weights without losing anything. More specifically, since we want to color all edges with two colors, the matching M is not allowed to pick up an edge from a 2-cycle, because in this case, we would need three colors. To avoid this, we could give both edges of a 2-cycle w' -weight zero. Then a maximum w' -weight Hamiltonian tour would have weight at least $L - b \cdot W$, since a Hamiltonian tour can run through only one edge of each 2-cycle. Consequently, a maximum w' -weight matching has weight at least $\frac{1}{2}(L - b \cdot W)$. However, we can only ensure that we can include one edge of each 2-cycle in each color class, in the worst case, the lighter one is put into both color classes. If we now give both edges in the 2-cycle w' -weight $2(b_i - c_i) \cdot W$ instead of zero, then a maximum w' -weight Hamiltonian tour has w' -weight at least $L - (2c + b) \cdot W$. This is an improvement, if b is strictly larger than c . If now the matching does not choose an edge of the 2-cycle, then everything is fine. But if it picks one up, then we now can include the heavier of the two edges into both color classes, thus we compensate the modified weight in this way. The scheme used in the algorithm also modifies the weights of the ingoing and outgoing edges of a 2-cycle in a similar fashion. The details will become clear in the course of the proof of Theorem 5.

2.2.1. The coloring lemma

The following lemma shows that the coloring described above always exists and that we can find one in polynomial time. If Z is a collection of node disjoint cycles in a directed graph $G = (V, E)$ (not necessarily covering G), then the process of *contracting* the cycles in Z produces a multigraph $H = (V', E')$ as follows: the node set V' consists of all nodes of V that do not belong to any cycle of Z and of one new vertex z_i for each cycle Z_i in Z . For each edge $e = (u, v) \in E$, we do the following: if both u and v belong to the same cycle, we discard e . Otherwise, we add the edge (u', v') to V' where $u' = u$ if u does not belong to a cycle, and $u' = z_i$ if u belongs to cycle Z_i . The node v' is defined in the same

manner. Note that E may be a multiset and hence H may be a multigraph, i.e., there may be more than one edge (in the same direction) between two nodes of H .

Lemma 2. *Let T be a collection of node disjoint 2-cycles on a node set V , P be a collection of node disjoint paths each with at least two edges, and M be a perfect matching on V . Moreover, assume that T and P are node disjoint and $T \cap M = \emptyset$. Let H be the multigraph obtained from the graph $G = (V, T \cup P \cup M)$ by contracting the cycles in T . Then the edges of the multigraph H can be colored with two colors such that the edges in each color class form a collection of node disjoint paths. Such a coloring can be found in polynomial time.*

Before we prove Lemma 2, we define another graph transformation: let G be a graph and Z be a collection of cycles. The process of *removing* the cycles in Z from G is defined in a similar manner as the contraction of cycles: here the node set V' consists of all nodes that do not belong to any cycle of Z . For each edge $e = (u, v) \in E$, we do the following: if neither u nor v are contained in any cycle of Z , then we add e to E' . All other edges are discarded. Furthermore, we add an additional edge (u, v) to E' if there is a path from u to v in the original graph G that has only nodes belonging to Z as internal nodes. (Note that the nodes of the connecting path can belong to more than one cycle of Z .) More generally, if Z is a collection of subgraphs of G , we define the process of *removing* the subgraphs of Z from G in the same manner.

To prove Lemma 2, we use the following “path coloring lemma” formulated by Kosaraju, Park, and Stein [9, Lemma 2]. We give a proof of it in Appendix A.

Lemma 3. *Let G be a directed loopless graph such that*

- (1) *all nodes of G have indegree and outdegree at most two and*
- (2) *all nodes of G have total degree at most three and*
- (3) *G does not contain any 2-cycles.*

Then the edges of G can be colored with two colors such that the edges in each color class form a collection of node disjoint paths. Moreover, such a coloring can be computed in polynomial time.

Proof of Lemma 2. We divide the edges in M into three classes:

- (1) An edge (u, v) in M is of the *first kind*, if (in the graph G) both nodes u and v belong to 2-cycles from T .
- (2) Edges (u, v) in M of the *second kind* are defined inductively: if (u, v) or (v, u) is in P , then (u, v) is an edge of the second kind. If u and v lie on the same path from P and all nodes between u and v on that path are only incident with edges of the second kind, then also (u, v) is an edge of the second kind. (See Fig. 1 for an illustration.)
- (3) All other edges are of the *third kind*.

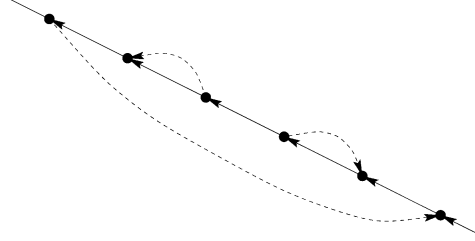


Fig. 1. Three edges of the second kind. (Solid edges represent path edges from P , dashed ones stand for matching edges from M .)

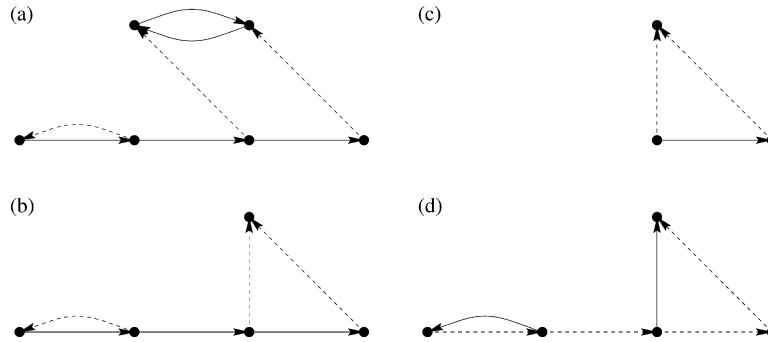


Fig. 2. An example of the coloring process: (a) The graph G . The solid edges in the upper part are the edges of T , the solid edges in the lower part are the edges of P . The dashed edges are the edges from M . (b) The multigraph H obtained by contracting the 2-cycle of T . (c) The maximal subpath of the second kind is removed. This yields H' . H' will now be colored via the path coloring lemma. (d) The final coloring: one color class is drawn dashed, the other one is drawn solid. The edges of the maximal subpath of the second kind get the same color as the other path edge.

The nodes of V inherit their type from the edges of M : a node v is of the same kind as the edge from M incident with v . (This is well-defined, because M is a perfect matching.) The nodes from T are either of the first or third kind. The nodes from P are either of the second or third kind.

A subpath s of a path from P is called a subpath of the second kind, if all nodes of s are of the second kind. The subpath s is called a maximal subpath of the second kind, if for any subpath s' of the second kind the fact that s is a subpath of s' implies $s = s'$. Let $S \subseteq P$ denote the collection of all maximal subpaths of the second kind.

We now contract the cycles from T and get the multigraph H . In H , we remove all paths from S to obtain a multigraph H' . To extend a coloring of H' to a coloring of H later on, it is useful to describe this process in detail. (Figure 2 illustrates this process.)

- (1) All subpaths s in S are removed (as defined above). If s contains the start or end node from a path from P , then this means that all nodes and edges of s and possibly one edge of P incident with the start or end node of s are discarded.
- (2) If s does not contain the start or end node of a path from P , then all nodes and edges of s and the two edges e and f of P incident with the start and end node of s are

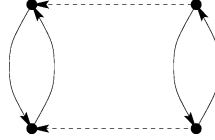


Fig. 3. Double edges emerging in H' . (Solid edges represent edges from T , dashed ones stand for edges of the first kind from M .) A 2-cycle would be created, if one of the edges from M is reversed.

discarded. Then an edge is inserted connecting the tail of e with the head of f . In other words, s is replaced by a single edge in H' .

- (3) All edges of the second kind from M are discarded.
- (4) All other edges remain in H' .

Our aim is to use Lemma 3 to color H' . The graph H' may still contain 2-cycles or double edges and thus not fulfill the premises of Lemma 3. But the 2-cycles and double edges of H' are of a very special form: no edge from M of the third kind can belong to a 2-cycle or double edge. So the only edges that can form double edges are two edges from M of the first kind each incident with the nodes from the same two 2-cycles from T (in G), see Fig. 3 for an illustration. Such 2-cycles and double edges are isolated, that is, no other edges from H' are incident with any nodes belonging to such a 2-cycle or double edge. Such an isolated 2-cycle or double edge can be trivially colored with two colors such that each color class forms a set of node disjoint paths. Moreover, such a coloring is independent of the coloring of the remaining parts of H' . Thus, we can color H' according to Lemma 3 with two colors such that each color class forms a set of node disjoint paths. Since the coloring from Lemma 3 can be found in polynomial time, the coloring of H' can also be computed in polynomial time.

Now we have to transfer the coloring of H' back to H . Any edge in H' that appears also in H is colored in H with the same color. The other edges are colored as follows:

- (1) If a subpath s of the second kind coincided with a complete path from P , then its edges are colored with one color (chosen arbitrarily) and the incident edges from M (which are of the second kind) are colored with the other color. If s contained only the start node of a path p from P but not the end node, then s is colored with the same color as the starting edge of p in H' . All incident edges from M are colored with the other color. The case where s contained the end but not the start node of p is treated completely alike.
- (2) If a subpath s of the second kind was replaced by a single edge, then all edges from s get the same color as this edge.
- (3) Edges from M of the second kind get the color that the corresponding subpath of the second kind does not have.

By construction, this translation does not create any cycles in one of the color classes. Thus, the coloring constructed above fulfills the assertions of the lemma. Moreover, it can be computed in polynomial time. $\square$

2.2.2. Approximation performance

We first prove that the each color class computed in step (7) of Algorithm 2 is indeed a collection of node disjoint paths.

Lemma 4. *Each color class computed in step (7) of Algorithm 2 consists of node disjoint paths.*

Proof. By Lemma 2, the edges of the multigraph H can be colored with two colors such that each color class consists of node disjoint paths. Then this coloring is transferred to the original graph $(V, P \cup \widehat{M})$, i.e., this colors the edges of $P \cup \widehat{M}$ such that each class consists of node disjoint paths. In step (7), we now add the edges of the 2-cycles to the color classes. It remains to show that after this, we still have two classes of node disjoint paths.

If a 2-cycle C_i is not in T , then one edge of C_i coincides with an edge of $M \setminus \widehat{M}$. This means that no edge of $P \cup \widehat{M}$ is incident with a node of C_i , because M is a matching. Thus, we can add the heavier edge of C_i to both color classes and create a new path of length one in both classes.

For a 2-cycle C_i in T , let e_i and f_i be the two edges from $\widehat{M}$ that are incident with the two nodes s_i and t_i for some 2-cycle C_i from T . If e_i and f_i have distinct colors, then inserting an edge from T obviously cannot create any new cycles. If e_i and f_i have the same color, then inserting an edge from T again cannot create any new cycle, since otherwise there would be a cycle in one of the color classes in the multigraph H . $\square$

The following theorem estimates the approximation performance of Algorithm 2.

Theorem 5. *Algorithm 2 returns a $(\frac{5}{8} + \frac{1}{8}b - \frac{1}{8}c + \frac{1}{8}x + \frac{1}{8}y - \frac{3}{8}z)$ -approximation to a Hamiltonian tour of maximum weight in G .*

Proof. Let L denote the w -weight of a Hamiltonian tour of maximum w -weight in G . What is the w' -weight L' of a maximum w' -weight tour in $(V, K(V), w')$? Figure 4 shows how a Hamiltonian tour in G can run through a 2-cycle C_i from C . In the first case, we loose weight $b_i \cdot W$ and get an additional weight $4(b_i - c_i) \cdot W$ under w' . In the second case,

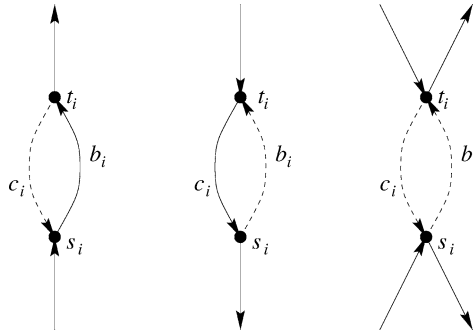


Fig. 4. The three ways how a Hamiltonian tour (drawn solid) can run through a 2-cycle.

we loose weight $c_i \cdot W$ and get an additional amount of $2(b_i - c_i) \cdot W$. In the third case, we loose nothing but get an additional $2(b_i - c_i) \cdot W$. Since $b_i \geq c_i$, $L' \geq L + (2b - 3c) \cdot W$. Hence, a matching M of maximum w' -weight has weight at least $\frac{1}{2}(L + (2b - 3c) \cdot W)$. On the other hand, the collection of paths P constructed in the algorithm has w -weight at least $(1 - b - c - z - p) \cdot W$.

How does this change after adding the 2-cycles in step (7) of the algorithm and when considering the weight function w instead of w' ? For each 2-cycle C_i not in T , we add the heavier edge to both color classes. Thus, we get an additional amount of $2b_i \cdot W$ and loose $2(b_i - c_i) \cdot W$, since we discard the edge of the matching. The total gain is $2c_i \cdot W$. (Note that since M is a matching, no edges from M can enter or leave C_i , thus we do not have to take care of these modified weights.)

Next we consider the edges from T and the edges from $\widehat{M}$ incident with nodes from T . It is convenient to use the following accounting principle: let $e = (u, v)$ be an edge of $\widehat{M}$. In order to have a modified weight, either $u = t_i$ for some $i \in I_2$ or $v = s_j$ for some $j \in I_2$ or both. In the first case, we account for the changed weight when considering C_i , in the second case when we look at C_j . In the third case, we split the loss: we charge the loss of $(b_i - c_i) \cdot W$ to C_i and of $(b_j - c_j) \cdot W$ to C_j . This allows us to treat all three cases in a unified way. For each C_i from T , let e_i be the edge of $\widehat{M}$ incident with s_i and f_i be the edge incident with t_i . If $e_i = (u, s_i)$ then we loose weight $(b_i - c_i) \cdot W$, but we get weight $b_i \cdot W$ back since we can add the heavier edge to one color class. If $e_i = (s_i, u)$, then we do not loose any weight and get $c_i \cdot W$, since we add the lighter edge to one color class. In the same way, if $f_i = (t_i, v)$ then we loose weight $(b_i - c_i) \cdot W$, but we get weight $b_i \cdot W$ back since we can add the heavier edge to one color class. Otherwise, we get weight $c_i \cdot W$. The total gain is $2c_i \cdot W$.

Overall, the total w -weight in both classes is at least $\frac{1}{2}(L + (2b - 3c) \cdot W) + (1 - b - c - z - p) \cdot W + 2c \cdot W$. Therefore, the heavier color class has w -weight at least $\frac{1}{4}L + (\frac{1}{2} - \frac{1}{4}c - \frac{1}{2}z - \frac{1}{2}p) \cdot W$. The claim of the theorem follows by plugging in $p \leq \frac{1}{4}(1 - b - c - x - y - z)$ and exploiting $W \geq L$. $\square$

2.3. Algorithm 3

To any cycle C_λ in C corresponds a second cycle obtained from C by reversing the direction of all edges participating in C_λ . In the case of a 2-cycle, we obtain the same cycle through this procedure. For longer cycles, however, we get a different cycle. Let $r_\lambda \cdot W$ denote the largest weight of an edge in the reversed cycle obtained from C_λ for $\lambda \in \{1, \dots, \ell\} \setminus I_2$.

In the case of a 3-cycle C_j , we have $r_j \leq x_j + y_j$, since otherwise we could replace C_j by a 2-cycle of weight at least $(r_j + z_j) \cdot W > w(C_j)$ and a single node and thus construct a cycle cover of larger weight, a contradiction. (Strictly speaking, this is not a cycle cover, since it contains a single node. If we added a dummy node to V , because the original graph G had an odd number of nodes, then every cycle cover necessarily contains an edge of weight zero. Thus, we can incorporate this single node into a cycle without any loss. If the original graph had an even number of nodes, then we simply add two dummy nodes.)

If r_j is large, then we replace a 3-cycle by a very unbalanced 2-cycle which is sometimes favorable. This is the key idea of Algorithm 3. In the following, let $r := \sum_{j \in I_3} r_j$. It turns

Input: a graph $G = (V, K(V), w)$.
Output: a Hamiltonian tour on G .

- (1) Compute a maximum weight cycle cover C of G .
- (2) Replace each 3-cycle in C by a 2-cycle of weight at least $r_j + z_j$ as described in Section 2.3. Extend this collection of cycles to a new cycle cover $\tilde{C}$ by forming new cycles arbitrarily with the remaining nodes.
- (3) Run Algorithm 2 with respect to the new cycle cover $\tilde{C}$.

Algorithm 3.

out that Algorithm 3 is inferior to Algorithm 2 if $r < z$. Thus, it is safe to assume that $r \geq z$ in the following. Furthermore, we can assume that the other (lighter) edges in the newly formed 2-cycles have total weight exactly $z \cdot W$, since reducing the weight will only reduce the overall weight.

Theorem 6. *Algorithm 3 returns a $(\frac{5}{8} + \frac{1}{8}b - \frac{1}{8}c - \frac{3}{8}x - \frac{3}{8}y - \frac{1}{8}z + \frac{1}{2}r)$ -approximation to a Hamiltonian tour of maximum weight in G .*

Proof. Let again L denote the weight of a Hamiltonian tour of maximum weight on G . The new cycle cover has weight at least $\tilde{W} = (1 - x - y + r) \cdot W$. The total weight of all heavier edges in the 2-cycles of $\tilde{C}$ is $\tilde{b} \cdot W$, where $\tilde{b} = b + r$. In the same way, the total weight of all lighter edges is $\tilde{c} \cdot W$, where $\tilde{c} = c + z$. Since we assume $r \geq z$, we have $\tilde{b} \geq \tilde{c}$. The sum of the weights of the lightest edges in all cycles of $\tilde{C}$ with at least four edges is $p \cdot W$.

Now we have to imitate the analysis of Algorithm 2. The weight of the matching $\tilde{M}$ is $\frac{1}{2}(L + (2\tilde{b} - 3\tilde{c}) \cdot W)$. Since we have removed the 3-cycles, the weight of the collection of paths $\tilde{P}$ is $(1 - b - c - x - y - z - p) \cdot W = (1 - \tilde{b} - \tilde{c} - x - y + r - p) \cdot W$. When inserting the 2-cycles, we get an additional amount of $2\tilde{c} \cdot W$. We come up with two color classes of node disjoint paths with total weight $\frac{1}{2}L + (1 - \frac{1}{2}\tilde{c} - x - y + r - p) \cdot W$. The color class with the larger weight contains at least weight $\frac{1}{4}L + (\frac{1}{2} - \frac{1}{4}\tilde{c} - \frac{1}{2}x - \frac{1}{2}y - \frac{1}{4}z + \frac{1}{2}r - \frac{1}{2}p) \cdot W$. Hence, Algorithm 3 returns a tour of weight at least $(\frac{5}{8} + \frac{1}{8}b - \frac{1}{8}c - \frac{3}{8}x - \frac{3}{8}y - \frac{1}{8}z + \frac{1}{2}r) \cdot W$. $\square$

In the extreme case $r = x + y$, Algorithm 3 yields a $(\frac{5}{8} + \frac{1}{8}b - \frac{1}{8}c + \frac{1}{8}x + \frac{1}{8}y - \frac{1}{8}z)$ -approximation, an improvement of $\frac{1}{4}z$ over Algorithm 2.

2.4. Algorithm 4

While Algorithm 3 is favorable when r is large, Algorithm 4 will be advantageous if r is small. Again, Algorithm 4 starts with computing a cycle cover C of maximum w -weight and a matching M of maximum w' -weight, where the weight function w' is defined as in Algorithm 2. Each 3-cycle in that cover either contains three nodes that are incident with three different edges from M (of the third kind) or two of the nodes are incident with

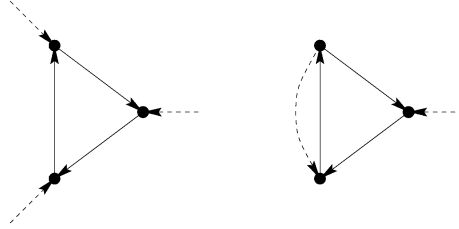


Fig. 5. The two possible configurations of a 3-cycle. (Solid lines are lines from the cycle, dashed ones from the matching.) Matching edges may also point in the opposite direction.

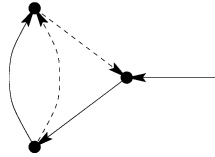


Fig. 6. The edge between two nodes of the second kind may be added to both color classes after discarding the matching edge. (One color class is represented by solid, the other by dashed lines.)

the same edge from M , that is, are nodes of the second kind. An illustration of these two possibilities is given in Fig. 5.

During the course of Algorithm 2, from each cycle of C with at least 3 edges, the lightest edge is discarded to obtain a collection of paths P . This is done to ensure that no loops occur after removing the subpaths of the second kind in Lemma 2. For example, in the 3-cycle on the right-hand side in Fig. 5, the left edge of the 3-cycle forms a maximal subpath of the second kind. After removing this path, we would obtain a loop. If we then translate the coloring back, we would get a cycle in one color class which we cannot deal with. On the other hand, the 3-cycle on the left-hand side in Fig. 5 can be colored according to Lemma 3. In this case, we get an additional amount of $z_j \cdot W$, if j denotes the index of the 3-cycle.

What can we do if the 3-cycle C_j contains two nodes of the second kind? After discarding the matching edge of the second kind in the 3-cycle, we can also color that 3-cycle according to Lemma 3. On the other hand, no matter how we color the edges of C_j , we can always add the edge of C_j between the two nodes of the second kind to each of the two color classes as is suggested in Fig. 6. If the matching edge coincides with one of the cycle edges, then we altogether get an additional amount of $z_j \cdot W$. If the matching edge points in the opposite direction, we lose $r_j \cdot W$ but get $2z_j \cdot W$ back. Since M is a matching of maximum w' -weight, $r_j \geq z_j$. Altogether, we get at least $(2z - r) \cdot W$ additional weight.

2.4.1. The coloring lemma

The following variant of Lemma 2 is used in Algorithm 4.

Lemma 7. Let T be a collection of node disjoint 2-cycles on a node set V , U a collection of node disjoint 3-cycles, P be a collection of node disjoint paths with at least

two edges, and M be a perfect matching on V . Let T , U , and P be pairwise node disjoint and $(T \cup U) \cap M = \emptyset$. Let H be the multigraph obtained from the graph $G = (V, T \cup U \cup P \cup M)$ by contracting the cycles in T . Then the edges of the multigraph H can be colored with two colors such that the edges in each color class form a collection of node disjoint paths. Such a coloring can be found in polynomial time.

Proof. We divide the edges in M into three classes in the same way as in the proof of Lemma 2. (Thus, edges from M incident with a node from a 3-cycle in U are of the third kind.)

The nodes of V inherit their type from the edges of M : a node v is of the same kind as the edge from M incident with v . The nodes from T are either of the first or third kind. The nodes from P are either of the second or third kind. The nodes from U are of the third kind.

Once more, let $S \subseteq P$ denote the collection of all maximal subpaths of the second kind (as defined in the proof of Lemma 2).

We contract the cycles from T and get the multigraph H . In H , we remove the paths from S in the graph to obtain a multigraph H' . By construction, this process does not affect U .

Our aim is to use Lemma 3 to color H' . The 3-cycles from U do not violate the premises of that lemma. However, the graph H' may still contain 2-cycles or double edges, but we have already demonstrated how to deal with them in the proof of Lemma 2.

Next we have to transfer the coloring of H' obtained via Lemma 3 back to H . Any edge in H that appears also in H' is colored in H with the same color. This determines the color of all edges in U . The remaining edges are colored as in the proof in Lemma 2.

By construction, this translation does not create any cycles in one of the color classes. Thus, the coloring constructed above fulfills the assertions of the lemma. Moreover, it can be computed in polynomial time. $\square$

2.4.2. Approximation performance

Theorem 8. Algorithm 4 returns a $(\frac{5}{8} + \frac{1}{8}b - \frac{1}{8}c + \frac{1}{8}x + \frac{1}{8}y + \frac{5}{8}z - \frac{1}{2}r)$ -approximation to a Hamiltonian tour of maximal weight in G .

Proof. Let L denote the w -weight of a heaviest tour in G . As in the proof of Theorem 5, the w' -weight L' of a heaviest tour in $(V, K(V), w')$ is at least $L - (2b - 3c) \cdot W$.

Thus, a matching M of maximum w' -weight has weight at least $\frac{1}{2}(L + (2b - 3c) \cdot W)$. The w -weight of the collection of paths P is at least $(1 - b - c - x - y - z - p) \cdot W$. For each 3-cycle in U , we get $(x + y + z) \cdot W$. But probably we have to change the direction of an edge from $M \setminus \widehat{M}$, this gives a total loss of at most $(r - z) \cdot W$. Taking care of the 2-cycles as in the proof of Theorem 5, we see that the total w -weight in both color classes is at least $\frac{1}{2}L + (1 - \frac{1}{2}c + z - p - r) \cdot W$.

Hence, the class with the larger w -weight has weight at least $\frac{1}{4}L + (\frac{1}{2} - \frac{1}{4}c + \frac{1}{2}z - \frac{1}{2}p - \frac{1}{2}r) \cdot W$. The claim of the theorem follows by plugging in $p \leq \frac{1}{4}(1 - b - c - x - y - z)$ and exploiting $W \geq L$. $\square$

Input: a graph $G = (V, K(V), w)$.
Output: a Hamiltonian tour on G .

- (1) Compute a maximum w -weight cycle cover C of G .
- (2) Define a new weight function w' on $K(V)$ as follows. For all $i \in I_2$, set $w'((s_i, t_i)) = w'((t_i, s_i)) = 2(b_i - c_i) \cdot W$. For all $i, j \in I_2, i \neq j$, set $w'((t_i, s_j)) = w((t_i, s_j)) + (b_i - c_i) \cdot W + (b_j - c_j) \cdot W$. For all $i \in I_2$, set $w'((u, s_i)) = w((u, s_i)) + (b_i - c_i) \cdot W$ and $w'((t_i, v)) = w((t_i, v)) + (b_i - c_i) \cdot W$, if $u \notin \{t_k \mid k \in I_2\}$ and $v \notin \{s_k \mid k \in I_2\}$. For all other edges e , set $w'(e) = w(e)$.
- (3) Compute a perfect matching M of maximum w' -weight on $(V, K(V), w')$.
- (4) Discard the lightest edge of each cycle with at least four edges in C to obtain a collection of node disjoint paths P .
- (5) Let T be the collection of all 2-cycles from C that do not share any edge with M .
- (6) Let U be the collection of all 3-cycles from C .
- (7) Let $\hat{M}$ be the set of all edges $(u, v) \in M$ such that neither (u, v) nor (v, u) is in $T \cup U$.
- (8) Let H be the multigraph obtained from $(V, T \cup U \cup P \cup \hat{M})$ by contracting the cycles in T . Color the multigraph H according to Lemma 7 and transfer the coloring back to $(V, U \cup P \cup \hat{M})$.
- (9) For each 2-cycle not in T , add the heavier edge to both color classes. For each 2-cycle C_i from T , let e_i and f_i be the two edges from $\hat{M}$ incident with s_i and t_i . If both e_i and f_i have the same color, then add the compatible edge of C_i to both color classes. Otherwise, add the edge of C_i compatible to e_i to the color class of e_i and the edge compatible to f_i to the color class of f_i .
- (10) For each edge $(u, v) \in M$ such that either (u, v) or (v, u) is in U : if $(u, v) \in U$, then add (u, v) also to the other color class, otherwise add (v, u) to the other class.
- (11) Patch together the paths of the color class with larger w -weight to obtain the Hamiltonian tour.

Algorithm 4.

2.5. Assembling the algorithms

The final algorithm works as follows: it runs Algorithms 1–4 on the given graph G and returns the heaviest tour produced by these four algorithms. The guaranteed approximation performance of this algorithm can be determined as follows:

$$\text{minimize } \max\{f_1, f_2, f_3, f_4\}, \quad (1)$$

where

$$\begin{aligned} f_1 &= \frac{3}{4} + \frac{1}{4}b - \frac{3}{4}c + \frac{1}{4}x + \frac{1}{4}y - \frac{3}{4}z, \\ f_2 &= \frac{5}{8} + \frac{1}{8}b - \frac{1}{8}c + \frac{1}{8}x + \frac{1}{8}y - \frac{3}{8}z, \\ f_3 &= \frac{5}{8} + \frac{1}{8}b - \frac{1}{8}c - \frac{3}{8}x - \frac{3}{8}y - \frac{1}{8}z + \frac{1}{2}r, \\ f_4 &= \frac{5}{8} + \frac{1}{8}b - \frac{1}{8}c + \frac{1}{8}x + \frac{1}{8}y + \frac{5}{8}z - \frac{1}{2}r \end{aligned}$$

over all choices of b, c, x, y, z , and r such that

$$b, c, x, y, z, r \geq 0, \quad b \geq c, \quad r \leq x + y, \quad b + c + x + y + z \leq 1, \quad x \geq y \geq z.$$

Since the number of critical points is not too large, this problem can be solved by hand and verified symbolically using a computer algebra system, see also Appendix B. It turns out that the above minimum equals $\frac{8}{13}$. It is attained for $b = \frac{3}{13}$, $c = \frac{3}{13}$, $x = \frac{5}{26}$, $y = \frac{5}{26}$, $z = \frac{2}{13}$, $r = \frac{4}{13}$ as well as $b = \frac{3}{13}$, $c = \frac{3}{13}$, $x = \frac{3}{13}$, $y = \frac{2}{13}$, $z = \frac{2}{13}$, $r = \frac{4}{13}$ (and all points in between). This proves the following theorem.

Theorem 9. *The heaviest of the four Hamiltonian tours produced by Algorithms 1–4 of G has at least $\frac{8}{13}(1 - 1/n)$ of the weight of a Hamiltonian tour of maximum weight of G .*

Note that it is not really necessary to run all four algorithms. Each algorithm starts with computing a cycle cover C . From this cover, we can compute the values of b , c , x , y , z , and r . These values determine which of the four algorithm has to be run to obtain the guaranteed approximation performance. For practical purposes, it can be useful to run all algorithms, since another of the four algorithms may nevertheless produce an even better approximation. The running time of our algorithm is dominated by the running time of computing a maximum weight bipartite matching and the time bound for finding a coloring as in Lemma 3, since the computation of the colorings in Lemmas 2 and 7 from the coloring in Lemma 3 can be implemented in quadratic time. In Appendix A, we present an implementation of the path coloring lemma of Kosaraju, Park, and Stein in cubic time. Thus, the total running time of our algorithm is $O(n^3)$ (say on a unit cost RAM).

3. Computing k -cycle covers

The tour produced by our algorithm can also be seen as an approximation of a maximum weight k -cycle cover, since a Hamiltonian tour surely is a k -cycle cover for any k . In the estimation of the approximation performance, however, we cannot conclude that the matching has half the weight of an optimum k -cycle cover in the modified graph but only a fraction of $(\tilde{k} - 1)/(2\tilde{k})$ where $\tilde{k} = k$ if k is odd and $k + 1$ otherwise. For instance, consider a graph where all edges have weight zero except a spanning collection of edges forming a set of node disjoint 3-cycles. Assume that the latter edges have weight one. Then an optimum 3-cycle cover has weight n , while a maximum weight matching only has weight $\frac{1}{3}n$. In the case of a 4-cycle cover, the worst case does not consist of a collection of 4-cycles, as they can be decomposed into two matchings, but of a collection of 5-cycles. Greater values of k are treated alike.

The analysis shows that the guaranteed approximation performances are (up to lower order terms) $\frac{9}{16}, \frac{25}{43}, \frac{25}{43}, \frac{49}{83}, \frac{49}{83}, \dots$ for $k = 3, 4, 5, 6, 7, \dots$. As k grows, the sequence of factors approaches $\frac{8}{13}$. For $k > n/2$, we finally achieve approximation ratio $\frac{8}{13}$. The precise performance ratios of the four algorithms are

$$f_1 = \frac{3}{4} + \frac{1}{4}b - \frac{3}{4}c + \frac{1}{4}x + \frac{1}{4}y - \frac{3}{4}z,$$

$$f_2 = \frac{3}{8} + \frac{\kappa}{2} + \left(\kappa - \frac{3}{8}\right)b - \left(\frac{5}{8} - \frac{3}{2}\kappa\right)c + \frac{1}{8}x + \frac{1}{8}y - \frac{3}{8}z,$$

$$f_3 = \frac{3}{8} + \frac{\kappa}{2} + \left(\kappa - \frac{3}{8}\right)b - \left(\frac{5}{8} - \frac{3}{2}\kappa\right)c - \frac{3}{8}x - \frac{3}{8}y - \left(\frac{5}{8} - \frac{3}{2}\kappa\right)z + \kappa r,$$

$$f_4 = \frac{3}{8} + \frac{\kappa}{2} + \left(\kappa - \frac{3}{8}\right)b - \left(\frac{5}{8} - \frac{3}{2}\kappa\right)c + \frac{1}{8}x + \frac{1}{8}y + \frac{5}{8}z - \frac{1}{2}r,$$

where $\kappa = (\tilde{k} - 1)/(2\tilde{k})$ with $\tilde{k}$ as defined above.

Appendix A. An algorithmic proof of Lemma 3

We provide an approach how to implement the path coloring lemma (Lemma 3) by Kosaraju, Park, and Stein [9] in polynomial time. We feel that this is necessary, since Kosaraju, Park, and Stein neither give a proof of existence of such a coloring nor a running time bound (although they use implicitly that it is polynomial time). To our knowledge, we here give the first rigorous proof of Lemma 3.

A.1. A first coloring

In the following, $G = (V, E)$ denotes the graph we want to color. First, we build an auxiliary undirected graph $A_1 = (E, F_1)$. The node set of A_1 is E , the edge set of the input graph G . In the following, whenever we speak of an edge, we refer to an element from E . If we view an edge from E as a node of A_1 , we will also speak of an A-node. The edges in F_1 and in further auxiliary graphs will be called A-edges. In the same way, when we speak of cycles and paths, we mean cycles and paths in G , while A-cycles and A-paths refer to cycles and path in some auxiliary graph. We define the edge set F_1 as follows: there is an A-edge $\{e, f\}$ in F_1 iff there are nodes u, v , and w in V such that $e = (u, v)$ and $f = (w, v)$ or $e = (v, u)$ and $f = (v, w)$. In other words, e and f have to be placed into different color classes, because their heads or tails meet in the node v . (For a directed edge (x, y) , x is its *tail* and y is its *head*.) Each A-node in A_1 has degree at most two. Hence, A_1 consists of simple A-cycles and A-paths. By construction, all A-cycles in A_1 have an even number of A-edges. This is due to the fact that an A-edge in F_1 corresponds to the event that either the heads or the tails of two edges from E meet in one node in G . This already shows that we can color the edges of G with two colors such that each of the two color classes consists of node disjoint paths and cycles. We just have to color the A-nodes in each A-path and A-cycle in an alternating way.

Lemma 10. *The edges of G can be colored with two colors such that each color class consists of node disjoint paths and cycles. Moreover, such a coloring can be found in polynomial time.*

We are left with eliminating the cycles. Note that for each A-cycle and A-path in A_1 , there are two possible colorings and we can choose one of them arbitrarily. This is the key observation.

A.2. Refining the coloring

In the following, let Z denote the set of all A-nodes that are contained in some A-cycle in A_1 . P stands for the set of all A-nodes that belong to A-paths with at least one A-edge in A_1 , i.e., the A-nodes in P are not isolated A-nodes. Finally, let S be the set of all isolated A-nodes in A_1 .

For an undirected path with nodes $v_1, \dots, v_\ell$, we call v_1 and v_ℓ the *end nodes* of this path. If the v_λ are A-nodes, then we call v_1 and v_ℓ A-end nodes.

Next we show how to color the edges from $G' := (V, Z \cup P)$ with two colors such that each color class contains only node disjoint paths. The only way how a cycle can arise in one color class of G' is shown in Fig. A.1. They are formed by edges that are all A-end nodes of A-paths in P . Since there are no 2-cycles in G , each such cycle consists of at least three edges.

Therefore, to avoid creating cycles in one color class, we have to ensure that two of the edges participating in such a cycle like in Fig. A.1 get different colors. Note that the cycles in $G' = (V, Z \cup P)$ are node disjoint; this is due to the degree restriction in G and the fact that each A-path in P has at least one A-edge in A_1 , that is, consists of at least two edges in G .

If say $e_1, \dots, e_\ell$ are the edges of a cycle in G' , we connect two consecutive ones of these edges in A_1 , i.e., we add say the A-edge $\{e_1, e_2\}$ to F_1 . Since the cycles are (edge and node) disjoint, there are at most $|E|/3$ many and we can find all of them in polynomial time. Let the resulting A-edge set be F_2 and let $A_2 = (Z \cup P, F_2)$. Compared to A_1 , the graph A_2 may have some new A-cycles. But these A-cycles all have an even number of A-edges, because adding the A-edge $\{e_1, e_2\}$ can be viewed as the event that two heads or two tails meet.

Lemma 11. *The edges of G' can be colored with two colors such that each color class consists of node disjoint paths. Moreover, such a coloring can be found in polynomial time.*

The above connection process has possibly created some new A-cycles. We transfer the corresponding A-nodes from P to Z . Since there is no danger of confusion here, we call the resulting sets again Z and P .

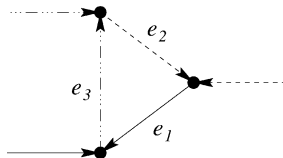


Fig. A.1. Paths from A_1 (drawn solid, dashed, and dashed-dotted) forming a cycle in G . The drawn nodes and edges are nodes and edges from G . Two such edges are connected by an A-edge if they are drawn in the same style (solid, dashed, and dashed-dotted). The edges outside the cycle may also point (simultaneously) in the opposite direction.

A.3. The final coloring

We are left with adding the edges from S to G' . We first consider a special case, to which we then reduce the general case. Assume that each cycle in G contains at least two edges from S . We color the edges of $Z \cup P$ according to the constraints already coded into A_2 . (Thereby, we choose one of the two possible colorings for each A-path or A-cycle in A_2 arbitrarily.) With respect to this coloring, we build the following auxiliary directed multigraph B : the A-nodes of B are the edges in S . The A-edges of B are colored with two colors. There is an A-edge (e, f) in B if there is a mono-colored path in G from the head of e to the tail of f . The A-edge (e, f) gets the same color as the corresponding path. If there are two edges $e = (u, v)$ and $f = (v, w)$ in S , then we add two A-edges from e to f in B , one for each color. Note that there cannot be any other edges incident with v , because otherwise e or f would not be in S .

We say that an A-cycle in B is mono-colored if all A-edges of that cycle have the same color. Since each cycle in G contains at least two edges from S , B cannot have any A-loops. Each A-node e in B has indegree and outdegree at most two. If two A-edges enter A-node of B , then they have different colors. The same is true for outgoing A-edges. In particular, any A-node in B lies on at most two A-cycles. By construction, the following statement holds.

Lemma 12. *If the A-nodes of B can be colored with two colors in such a way that on each mono-colored A-cycle in B , at least one A-node has a different color than the color of the A-cycle, then G can be colored with two colors such that each color class consists of node disjoint paths.*

To B , we define the undirected multigraph B^* as follows: the node set of B^* are the mono-colored A-cycles in B and there is an edge between two such cycles if they have an A-node in common. Any connected component in B induces a connected component in B^* and vice versa. We call an A-node of B *free*, if it lies on at most one mono-colored A-cycle in B . We have the following lemma.

Lemma 13. *Each connected component in B either has a free A-node or the induced component in B^* contains a cycle or double edge.*

Proof. Since B does not have any A-loops, each connected component in B without a free A-node necessarily induces a component in B^* with a cycle or a double edge. $\square$

The following lemma shows that a coloring as in Lemma 12 always exists and can be found in polynomial time.

Lemma 14. *The A-nodes of the graph B can be colored with two colors such that on each mono-colored A-cycle in B at least one A-node has the color other than the A-cycle. Moreover, such a coloring can be found in polynomial time.*

Proof. First, we discard all A-edges of B that do not lie on any mono-colored cycle in B . Furthermore, it is sufficient to prove the lemma for each connected component of B separately.

If the component has a free A-node, then we give this A-node the color other than the A-cycle it belongs to. We then remove this A-cycle. This creates a number of connected components. Each of them necessarily has a free A-node (the one which laid on the removed A-cycle) and hence we can go on recursively.

If the component does not have a free A-node, then by Lemma 13, B^* has a cycle or double edge (which we view as a cycle with two edges). By construction, each cycle in B^* has an even number of edges. If we color the edges (which are A-nodes in B) of this cycle in B^* in an alternating fashion, then each of the involved A-cycles in B will have an A-node of the other color. Hence we may remove all these A-cycles. We get a number of connected components in B . Each of them necessarily has a free node (the one which laid on a removed A-cycle) and hence we are in the first case. $\square$

Finally we remove the assumption that each cycle in the graph G has at least two edges from S .

Lemma 15. *The edges of G can be colored with two colors such that each color class consists of node disjoint paths. Moreover, such a coloring can be found in polynomial time.*

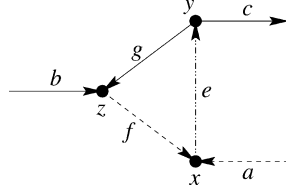
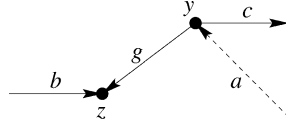
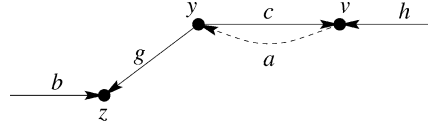
Proof. The proof is by induction in $|V|$. If $|V| \leq 3$, then such a coloring can be easily found.

Assume that $|V| \geq 4$. We construct the graphs A_1 , A_2 , B , and B^* as described above. We first treat some trivial cases: if there is an edge $e = (u, v)$ in S such that no other edge in E is incident with u or v , then we can give e any color. Hence we may remove e from the subsequent considerations. If there are edges (u, v) and (v, w) in S , then no other edge in E can be incident with v , because both edges are in S . We give them different colors and may ignore them, too. Let the resulting set of isolated A-nodes be T .

For each $t \in T$ we check if the tail of t is reachable from the head of t in $(V, Z \cup P)$. If this is not the case, then each cycle in $(V, Z \cup P \cup T)$ contains at least two edges from T . Thus by Lemmas 12 and 14, we can color $(V, Z \cup P \cup T)$ with two color such that each color class consists of node disjoint paths. As described above, this coloring can be extended to a coloring of $G = (V, Z \cup P \cup S)$.

Figure A.2 shows a typical cycle with only one edge from T . Such a cycle consists of one edge e from T and has at most one edge which is not an A-end node of some A-path in A_2 (here this edge is g). Since there are no 2-cycles, either the edge directly before e or directly after e is an A-end node of some A-path in A_2 . In Fig. A.2, this edge is f . The case where this edge follows e is symmetric. The cycle may have more A-end nodes of some A-paths in A_2 , this situation is settled in the same fashion. We treat this case later on.

We discard the edges e and f as well as the node x and let the edge a point to y instead. Let the resulting graph be $\hat{G}$ (see Fig. A.3 for an illustration). From A_2 , we obtain the corresponding graph $\hat{A}_2$ by discarding the A-nodes e and f as well as the A-edge $\{a, f\}$. The graph $\hat{G}$ fulfills the degree restrictions of the path coloring lemma.

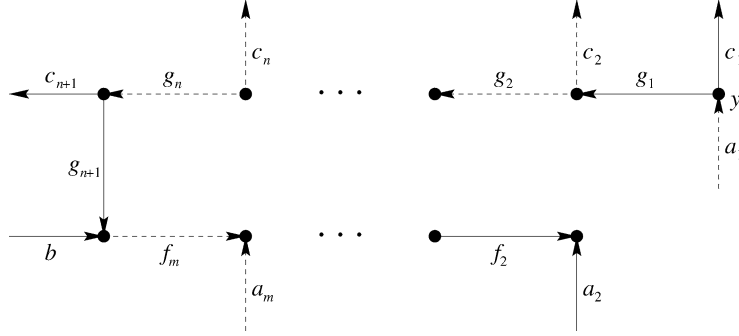
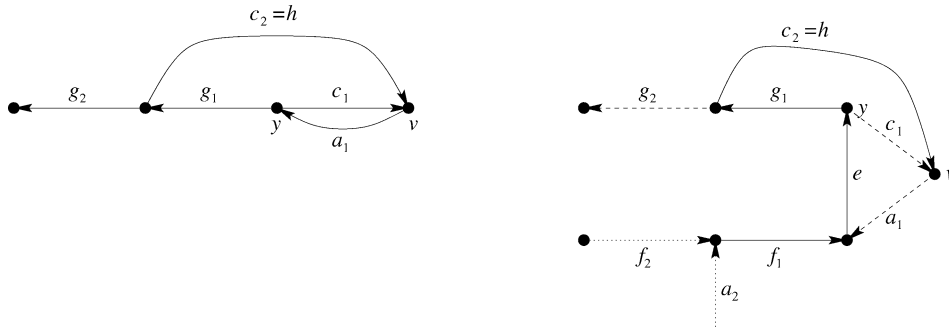
Fig. A.2. A cycle with three edges containing only one edge e from T .Fig. A.3. Discarding the edges e and f yields the new graph $\hat{G}$.Fig. A.4. A 2-cycle can arise. In this case, a is an isolated A-node in $\hat{A}_2$ and is simply discarded.

If $\hat{G}$ does not contain any 2-cycles (this is equivalent to the fact that the tail of a and the head of c are different), then we can apply the induction hypothesis. If the edge a is an isolated A-node in $\hat{A}_2$, then we remove it from P and add it to T . If a is not in T , then it might be possible that there is (exactly) one cycle in $\hat{G}$ that runs through a and c and does not contain any edges from T . In this case, c is an A-end node of some A-path. As in the previous section (when building A_2), we also add the A-edge $\{c, a\}$ to $\hat{A}_2$. By the induction hypothesis, there is a coloring of $\hat{G}$ with two colors such that each color class consists of node disjoint paths. This coloring can be extended to a coloring of $(V, Z \cup P \cup T)$ by giving e the same color as a and f the other color. This cannot create a cycle, because otherwise there would be a cycle in $\hat{G}$ running through a and c . As described above, we can extend the coloring of $(V, Z \cup P \cup T)$ to a coloring of $G = (V, Z \cup P \cup S)$.

If the head of c and the tail of a are identical, then a 2-cycle arises. To be specific, call this node v . Depending on the direction of the third edge h incident with v , we do the following: if v is the head of h (or h is not existent at all), then a is an isolated edge, see Fig. A.4. We delete a , apply the induction hypothesis and give a the color c has not. Then there is no cycle running through a , because a and c have different colors and there is no way out of z .

The case where v is the tail of h splits into two subcases, namely $b \neq h$ and $b = h$. In the first case (depicted in Fig. A.5), we discard the edges c and a and identify the nodes y and v . This cannot create any 2-cycles or double edges, as $b \neq h$. To $\hat{A}_2$, we also add the A-edge $\{g, h\}$. We apply the induction hypothesis and then give c the color of h and a the color of g . Obviously, no cycle can arise.

If finally $b = h$, then due to the degree restriction, we have encountered an isolated component of G . It is colored as shown in Fig. A.6.

Fig. A.8. Discarding the edges e and f_1 yields the new graph $\widehat{G}$.Fig. A.9. Left-hand side: removing the 2-cycle formed by a_1 and c_1 creates a loop. Right-hand side: the coloring of the discarded edges. One color class is drawn solid, the other one dashed. Dotted edges can have either color. (But the colors of f_2 and a_2 are always distinct.)

The case that v is the head of h is more complicated. Here we remove the 2-cycle formed by a_1 and c_1 , that is, we discard the edges g_1 , a_1 , c_1 , and h as well as the nodes y and v and add an edge d pointing from the tail of h to the head of g_1 . If this does not create a loop or 2-cycle, then we color this graph via the induction hypothesis. We then give g_1 , a_1 , and h the color of d and c_1 the other color. This cannot create a cycle.

If removing the 2-cycle formed by a_1 and c_1 yields a loop, then c_2 and h coincide (see Fig. A.9, left-hand side). In this case, we go back to the original graph G and discard the edges g_1 , c_1 , c_2 , e , f_1 , and a_1 (and all nodes that are isolated thereafter). We color the remaining graph inductively and then add the discarded edges as shown on the right-hand side of Fig. A.9. The color of g_2 determines the color of the discarded edges. Note that f_2 and a_2 can be colored in two ways, but this does not affect f_1 . Adding the discarded edges does not create any cycles, since g_1 and g_2 have distinct colors.

Finally, we have to treat the case that another 2-cycle occurs after removing the 2-cycle formed by a_1 and c_1 . In this case, $n \geq 2$ and c_3 and h coincide. The left-hand side of Fig. A.10 illustrates this situation. Here we discard the edges g_2 , g_1 , h , c_1 , and a_1 and identify the tail of g_3 with the tail of c_2 (see Fig. A.9, right-hand side). We color the resulting graph by applying the induction hypothesis. (If $n = 2$ and $c_2 = b$, then this graph

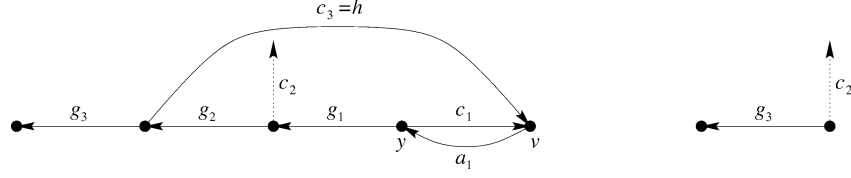


Fig. A.10. Left-hand side: removing the 2-cycle formed by a_1 and c_1 creates another 2-cycle. Right-hand side: the reduced graph to be colored.

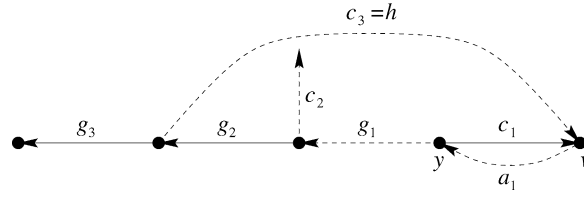


Fig. A.11. The coloring of the discarded edges. One color class is drawn solid, the other one dashed.

has a double edge formed by c_2 and g_3 . The tails of c_2 and g_3 do not belong to a third edge by construction, thus we can safely give these two edges two different colors.) Identifying the tail of g_3 with c_2 ensures that g_3 and c_2 get different colors. The discarded edges are then colored as depicted in Fig. A.11. $\square$

For estimating the running time, note that the number of edges $|E|$ in G is at most $\frac{3}{2}|V|$. The total running time is dominated by the running time of the algorithm presented in Lemma 15. Since the number of nodes is reduced by at least one node when building $\widehat{G}$, the induction hypothesis is applied at most $|V|$ times. In each such step, we test for $|T| \leq |E|$ edges whether they lie on a cycle. Each test can be done in time $O(|E|)$ using depth first search. Thus, the total running time is $O(n^3)$. There might be some room for improvement, since we test each edge in T in each application of the induction hypothesis. This seems to be necessary, since one edge from T is discarded in each such step and new cycles with only one edge in T could arise. It may nevertheless be possible to reuse some previous computations and to improve the running time in this way. Dynamic graph algorithms may hold the key.

Appendix B. Estimating the approximation performance

We outline how to solve (1). First, note that we can replace $x + y$ by a single variable u . Furthermore, since the coefficient of b is always positive and the coefficient of c is always negative, $c \leq b$ enforces $c = b$ when the minimum is attained. Thus, we can rewrite $f_1, \dots, f_4$ as

$$\begin{aligned} f_1 &= \frac{3}{4} - \frac{1}{2}c + \frac{1}{4}u - \frac{3}{4}z, \\ f_2 &= \frac{5}{8} + \frac{1}{8}u - \frac{3}{8}z, \end{aligned}$$

$$f_3 = \frac{5}{8} + \frac{3}{8}u - \frac{1}{8}z + \frac{1}{2}r,$$

$$f_4 = \frac{5}{8} + \frac{1}{8}u + \frac{5}{8}z - \frac{1}{2}r.$$

Only f_3 and f_4 depend on r . The coefficient of r in f_3 is positive, in f_4 , it is negative. For fixed u and z , intersecting (the hyperplanes given by) f_3 and f_4 yields

$$r = \frac{1}{2}u + \frac{3}{4}z. \quad (\text{B.1})$$

This is feasible, since $\frac{1}{2}u + \frac{3}{4}z \leq u$. We replace f_3 and f_4 by their intersection

$$f'_3 = \frac{5}{8} - \frac{1}{8}u + \frac{1}{4}z.$$

Next we examine f_2 and f'_3 , which both do not depend on c . The set of feasible points is given by $0 \leq z \leq \frac{1}{2}u$ and $u + z \leq 1$. Checking the points $(1, 0)$, $(0, 1)$, and $(\frac{1}{3}, \frac{2}{3})$ for (z, u) , we see that $\max\{f_2, f'_3\}$ is at least $\frac{5}{8}$ at these points. (It is easy to see that the guaranteed approximation performance is at most $\frac{5}{8}$ by setting $b = c = \frac{1}{2}$ and the other variables to zero.) Intersecting f_2 with f'_3 , we get

$$u = \frac{5}{2}z. \quad (\text{B.2})$$

This is again feasible, since $2z \leq \frac{5}{2}z$. We substitute (B.2) into f_1 and f_2 and obtain

$$f'_1 = \frac{5}{8} - \frac{1}{16}z,$$

$$f'_2 = \frac{3}{4} - \frac{1}{2}c - \frac{1}{8}z.$$

The set of feasible points is now given by $0 \leq u + z = \frac{7}{2}z \leq 1 - 2c$ and $0 \leq c \leq \frac{1}{2}$. At the points $(\frac{2}{7}, 0)$, $(0, 0)$, and $(0, \frac{1}{2})$ for (z, c) , $\max\{f'_1, f'_2\}$ is always at least $\frac{5}{8}$. We intersect f'_1 and f'_2 and obtain

$$z = 2 - 8c. \quad (\text{B.3})$$

Now the minimum is attained at the intersection of (B.3) with $z = \frac{2}{7} - \frac{4}{7}c$, which is the boundary of the feasible region. This finally yields $c = \frac{3}{13}$. Plugging this into (B.3), (B.2), and (B.1) yields $z = \frac{2}{13}$, $u = \frac{5}{13}$, and $r = \frac{4}{13}$.

References

- [1] G. Ausiello, P. Crescenzi, G. Gambosi, V. Kann, A. Marchetti-Spaccamela, M. Protasi, Complexity and Approximation, Springer-Verlag, 1999.
- [2] A. Blum, T. Jiang, M. Li, J. Tromp, M. Yannakakis, Linear approximation of shortest superstrings, J. ACM 41 (4) (1994) 630–647.
- [3] R. Carr, S. Vempala, Towards a $4/3$ approximation for the asymmetric traveling salesman problem, in: Proc. 11th Ann. ACM–SIAM Symp. on Discrete Algorithms (SODA), 2000, pp. 116–125.

- [4] N. Christofides, Worst-case analysis of a new heuristic for the travelling salesman problem, in: J.F. Traub (Ed.), *Algorithms and Complexity: New Directions and Recent Results*, Academic Press, New York, NY, 1976, p. 441.
- [5] M.L. Fisher, L. Nemhauser, L.A. Wolsey, An analysis of approximations for finding a maximum weight Hamiltonian circuit, *Oper. Res.* 27 (1979) 799–809.
- [6] A.M. Frieze, G. Galbiati, F. Maffioli, On the worst-case performance of some algorithms for the asymmetric traveling salesman problem, *Networks* 12 (1) (1982) 23–39.
- [7] R. Hassin, S. Rubinstein, Better approximations for maxTSP, *Inform. Proc. Letters* 75 (4) (2000) 251–258.
- [8] R. Hassin, S. Rubinstein, A $\frac{7}{8}$ -approximation algorithm for metric MaxTSP, in: *Proc. 7th Int. Workshop on Algorithms and Data Structures (WADS)*, Lecture Notes in Comput. Sci., Vol. 2125, 2001, pp. 205–209.
- [9] S. Rao Kosaraju, J.K. Park, C. Stein, Long tours and short superstrings, in: *Proc. 35th Ann. IEEE Symp. on Foundations of Comput. Sci. (FOCS)*, 1994.
- [10] A.V. Kostochka, A.I. Serdyukov, Polynomial algorithms with the estimates $\frac{3}{4}$ and $\frac{5}{6}$ for the traveling salesman problem of the maximum, *Upravlyaemye Sistemy* 26 (1985) 55–59 (in Russian).
- [11] M. Lewenstein, M. Sviridenko, Approximating maximum asymmetric TSP, in: *Proc. 14th ACM–SIAM Symp. on Discrete Algorithms (SODA)*, 2003, pp. 646–655.
- [12] C.H. Papadimitriou, M. Yannakakis, The traveling salesman problem with distances one and two, *Math. Oper. Res.* 18 (1993) 1–11.
- [13] A.I. Serdyukov, An algorithm with an estimate for the traveling salesman problem of the maximum, *Upravlyaemye Sistemy* 25 (1984) 80–86 (in Russian).
- [14] Z. Sweedyk, A $2\frac{1}{2}$ -approximation algorithm for shortest superstring, *SIAM J. Comput.* 29 (3) (1999) 954–986.
- [15] J. Tarhio, E. Ukkonen, A greedy approximation algorithm for constructing shortest common superstrings, *Theoret. Comput. Sci.* 57 (2) (1988) 131–145.
- [16] J.S. Turner, Approximating algorithms for the shortest common superstring problem, *Inform. Comput.* 83 (1) (1989) 1–20.